

Architecting a Zero-Cost Trading Bot: A Framework for Speed and Accuracy Using Distributed Apple Silicon and Free Cloud Services

Architectural Blueprint for a Distributed Local Cluster

The foundation of a high-performance, cost-free trading bot lies in its architectural design, which must intelligently orchestrate available resources to achieve maximum throughput and minimal latency. The user's requirement to leverage two local MacBooks—specifically an M1 with 8GB of RAM and an M2 with 16GB of RAM—necessitates a distributed computing approach [58](#). This section outlines a comprehensive architectural blueprint that transforms these two consumer-grade devices into a powerful, heterogeneous computing cluster. The strategy involves a clear division of labor, robust communication protocols managed by open-source tools, and a logical separation of data ingestion, computational processing, and trade execution tasks. The central challenge is not merely to connect the two machines but to create a seamless workflow where each component operates at peak efficiency, directly addressing the goal of rapid analysis and decision-making. This architecture eschews paid commercial middleware in favor of open-source solutions, ensuring adherence to the strict "absolute free" constraint while building a scalable and maintainable system. The proposed design prioritizes minimizing data transfer bottlenecks and maximizing parallel processing capabilities, creating a framework capable of handling the demanding computational load of modern quantitative trading strategies without relying on external paid infrastructure.

A successful distributed architecture for this project hinges on a strategic division of labor that capitalizes on the distinct hardware profiles of the M1 and M2 MacBooks. The MacBook equipped with the M2 chip and 16GB of RAM should be designated as the Central Processing Node (CPN). This machine possesses superior computational power, including higher theoretical floating-point operations per second (FLOPS) and greater memory bandwidth compared to its M1 counterpart [58](#). Its responsibilities will encompass the most resource-intensive aspects of the trading operation. First, it will serve as the primary orchestrator of the distributed computing cluster, managing the overall workflow and task distribution. Second, it will handle the initial data ingestion, pulling

market data from various exchanges via API calls. Third, it will run the computationally expensive process of training the predictive machine learning models. Finally, it will aggregate results from simulations performed by the worker nodes and execute the final trade selection logic. This centralized control ensures that complex, overarching decisions are made by the most capable machine in the cluster, reducing the risk of errors or inconsistencies that could arise from decentralized processing.

Conversely, the MacBook with the M1 chip and 8GB of RAM will function as a dedicated Worker Node (WN). While its hardware is less potent, particularly concerning its limited RAM, it remains a valuable asset for parallel computation. Its primary role will be to execute discrete, computationally intensive tasks assigned by the CPN. These tasks will typically involve running individual scenarios within a larger backtest or performing parallel predictions on different segments of a time-series dataset. By offloading these specific jobs to the M1, the M2 is freed up to focus on orchestration, model training, and final decision-making, thereby optimizing the use of both machines' processing cycles. This client-worker model, facilitated by an open-source library such as Dask, creates an efficient pipeline where the CPN acts as the manager and the WN acts as a specialized assistant, working in concert to solve a large problem by breaking it down into smaller, manageable pieces [50](#). This structure is fundamental to achieving the desired speed-up through parallelization.

The communication and coordination between the Central Processing Node and the Worker Node are critical to the system's performance and stability. The choice of technology for this inter-node communication must be lightweight, reliable, and entirely free. The Python library Dask is the ideal tool for this purpose, as it is explicitly designed to create distributed computing clusters from multiple machines, either locally or across a network [50](#). Dask provides a familiar interface for users already working in the Python data science ecosystem, allowing the same codebase used for single-machine development to be scaled out to a multi-node cluster with minimal modification. The setup involves installing Dask on both MacBooks and configuring the M2 as the scheduler—a central point of control—and the M1 as a worker node that connects to the scheduler. When a task, such as a backtest simulation, is initiated on the M2, the Dask client breaks it down into smaller units of work, which are then dispatched to the available workers. The results are collected and aggregated by the scheduler, providing a transparent and efficient parallel processing mechanism. This approach eliminates the need for custom-built messaging systems or costly commercial enterprise service buses, aligning perfectly with the project's zero-cost mandate.

To ensure the entire system operates seamlessly, a logical separation of concerns is essential. The architecture can be broken down into three primary layers: the Data Layer,

the Computational Layer, and the Execution Layer. The Data Layer is responsible for acquiring and preprocessing market data. This layer will be hosted on the Central Processing Node (M2), which will periodically fetch data from cryptocurrency exchange APIs. This data will be stored in a structured format, such as Parquet files or a local SQLite database, making it easily accessible for subsequent processing. The Computational Layer is the heart of the system, residing primarily on the Central Processing Node but utilizing the Worker Node (M1) for parallel tasks. This layer runs the core logic of the trading bot, including predictive modeling and scenario simulation. It uses the Dask framework to distribute computational workloads across both machines, significantly accelerating tasks like backtesting thousands of historical scenarios or evaluating the impact of different parameters on a trading strategy. The Execution Layer is also managed by the Central Processing Node. Once the Computational Layer has identified an optimal trade based on its analysis, the Execution Layer is responsible for placing the corresponding order on a supported cryptocurrency exchange. This is done programmatically by interfacing with the exchange's API, a task that requires careful management of API keys and rate limits but does not necessitate the involvement of the Worker Node. This layered architecture ensures modularity, making the system easier to debug, maintain, and scale in the future. For instance, if a new type of predictive model needs to be tested, only the Computational Layer needs to be modified, without affecting the Data or Execution Layers.

Finally, the entire operational workflow can be automated and managed using version control and continuous integration principles, further enhancing the system's reliability and efficiency. All source code for the trading bot, including the data ingestion scripts, predictive models, and simulation logic, should be stored in a Git repository hosted on a platform like GitHub. This allows for easy tracking of changes and collaboration. To automate key processes, such as scheduled data collection, a CI/CD (Continuous Integration/Continuous Deployment) pipeline can be established using GitHub Actions [35 44](#). A workflow can be configured to trigger a script every few minutes or hours, instructing the bot to pull the latest market data. This frees the local machines from running constant background processes and ensures that the data used for analysis is always up-to-date. Similarly, workflows can be set up to automatically run backtests whenever new code is pushed to the main branch of the repository, providing immediate feedback on whether a recent change has degraded the strategy's performance. By combining a well-defined distributed architecture with robust automation, the system becomes not just a powerful analytical tool, but also a self-sustaining, highly optimized engine for quantitative trading, fully compliant with the constraints of being built entirely with free resources.

Component	Role	Machine Designation	Key Responsibilities
Central Processing Node (CPN)	Orchestrator & Primary Workhorse	MacBook Pro (M2, 16GB RAM)	Manages Dask cluster, ingests raw market data, trains ML models, executes final trade logic, aggregates simulation results. 58
Worker Node (WN)	Parallel Task Executor	MacBook Air (M1, 8GB RAM)	Executes discrete, computationally intensive tasks (e.g., individual backtest scenarios) assigned by the CPN. 58
Communication & Coordination	Inter-node Messaging	Dask Library (Open Source)	Facilitates task distribution and result aggregation between the CPN and WN, enabling parallel processing. 50
Data Layer	Data Acquisition & Storage	Central Processing Node (M2)	Fetches data from exchange APIs and stores it in an organized local format (e.g., Parquet, SQLite).
Computational Layer	Core Analytics Engine	Both Machines (Dask Cluster)	Runs predictive models and parallelized simulations using data from the Data Layer. 59
Execution Layer	Trade Placement	Central Processing Node (M2)	Interfaces with the exchange API to place orders based on signals generated by the Computational Layer.

This architectural blueprint provides a clear, actionable, and cost-free framework for constructing a sophisticated trading bot. By assigning complementary roles to the two MacBooks and leveraging the power of open-source distributed computing, the system is designed to meet the demanding requirements of speed and accuracy in the fast-paced cryptocurrency markets.

Leveraging Apple Silicon Hardware for High-Speed Computation

To achieve the ambitious goals of speed and predictive accuracy, the trading bot's computational core must be finely tuned to exploit the unique architecture of the Apple M1 and M2 silicon chips. These System-on-Chips (SoCs) represent a significant departure from traditional x86 architectures, offering substantial performance advantages for specific workloads common in quantitative finance, such as numerical computations, matrix operations, and machine learning inference [58](#). A deep understanding of their capabilities—namely the unified memory architecture, superior CPU and GPU performance, and specialized Neural Engine—is paramount to designing an efficient and powerful local processing environment. This section delves into the specific characteristics of Apple's M-Series SoCs and provides a detailed guide on how to harness their full potential, transforming the user's two MacBooks into a formidable, albeit constrained, high-performance computing cluster. The analysis focuses on maximizing

computational throughput while navigating the inherent limitations, such as the 8GB RAM on the M1 model, to build a system that is both fast and stable.

The defining feature of Apple's M1, M2, and subsequent silicon is the unified memory architecture [58](#). Unlike conventional computers where the CPU and GPU have separate pools of memory connected by a slower bus, Apple's SoCs integrate the system memory (LPDDR4X/LPDDR5/LPDDR5X) directly onto the same chip as the processor cores. This shared pool of high-bandwidth memory is accessible by the CPU, GPU, Neural Engine, and other components with extremely low latency [58](#). For a trading bot, this translates directly into faster data access and more efficient computation. When a task requires data to be processed by both the CPU and GPU, there is no need to copy it between separate memories, eliminating a major source of overhead and delay. This is particularly beneficial for machine learning tasks, where large datasets and model weights must be frequently accessed during training and prediction phases. The memory bandwidth, a critical metric for performance, varies significantly across the series: the M1 achieves up to 67 GB/s, while the M2 reaches 100 GB/s, and the latest M4 can achieve 120 GB/s [58](#). This means that for data-intensive operations, the M2-powered MacBook will have a considerable advantage over the M1 model simply due to its ability to move data around the chip more quickly. However, even the M1's 67 GB/s is substantially higher than what is typically found in comparable Intel-based laptops, making both machines well-suited for this type of workload.

Beyond memory, the raw computational power of the CPUs and GPUs in the M-Series chips offers significant benefits. The M2 chip features a more advanced CPU design that delivers a higher range of theoretical peak single-precision floating-point operations per second (FP32 FLOPS) compared to the M1, ranging from 2.86 to 3.57 TFLOPS versus the M1's 2.29 to 2.61 TFLOPS [58](#). This increase in raw computational throughput is crucial for tasks like complex backtesting, Monte Carlo simulations, and training sophisticated statistical models. More importantly, starting with the M2, Apple's GPUs have become significantly more powerful than their CPUs for certain types of parallelizable mathematical operations [58](#). Tasks involving large-scale matrix multiplication and convolutions—foundations of many machine learning algorithms—are now better suited for the GPU. This allows developers to offload these intensive calculations from the CPU, freeing it up for other tasks and dramatically increasing the overall speed of the application. The M4 GPU can peak at 2.9 TFLOPS, showcasing the continued investment Apple is making in its graphics processing capabilities [58](#). For the trading bot, this means that computationally heavy parts of the predictive models can be accelerated by running them on the GPU, provided the underlying software supports it.

The GPU's computational performance is unlocked through Apple's Metal API, a low-level framework for graphics and compute tasks [58](#). Higher-level libraries often provide abstractions over Metal, such as Metal Performance Shaders (MPS), which offers a suite of pre-optimized kernels for common operations like matrix math and convolution [58](#). This is where the synergy between the hardware and the chosen software stack becomes critical. To leverage the M2's powerful GPU, the trading bot's core algorithms must be implemented using frameworks that can target the Metal API. This is a key reason why selecting libraries like XGBoost, which has explicit support for GPU acceleration, is so important [59](#). When the `gpu_hist` tree method in XGBoost is used, the library automatically offloads the most intensive parts of the gradient boosting process to the GPU, utilizing Metal under the hood to perform calculations in parallel across thousands of cores [58](#) [59](#). This can lead to dramatic speedups, especially when dealing with large datasets, directly contributing to the bot's goal of analyzing data "faster and quicker." The M1, while having a capable GPU, does not see the same degree of GPU-vs-CPU disparity as the M2, but it still benefits immensely from Metal-accelerated computations.

In addition to the CPU and GPU, the Apple Neural Engine (ANE) is a specialized hardware accelerator present in all M-series chips, designed specifically for machine learning workloads [58](#). The ANE is composed of 16 specialized cores optimized for tensor-based computations, particularly at low precision formats like INT8 and FP16 [58](#). It lacks native support for 32-bit floating-point (FP32) operations, which makes it unsuitable for training complex models that require high numerical precision. However, for the inference phase—the process of using a trained model to make predictions—it is exceptionally efficient. After a model like XGBoost has been trained on the M2's GPU, the final step of applying that model to incoming market data can be offloaded to the Neural Engine. This would allow the bot to generate predictions almost instantaneously, consuming very little power in the process. While implementing ANE-specific code may require additional effort beyond standard ML libraries, it represents a frontier for achieving ultra-low-latency predictions on local hardware. For the current scope of the project, focusing on GPU acceleration for training and general-purpose computation is the most pragmatic approach, but awareness of the ANE's capabilities is important for future optimization.

However, the hardware is not without its constraints, and these must be carefully managed to ensure system stability. The most significant limitation in this setup is the 8GB of RAM on the older MacBook (M1) [58](#). In a distributed computing environment managed by Dask, this machine will act as a worker node, but its memory capacity will dictate the size of the tasks it can handle. Attempting to assign it a task that requires loading a dataset larger than its available RAM will lead to swapping to disk, causing

catastrophic slowdowns and potentially crashing the process. Therefore, the data must be partitioned appropriately before being sent to the M1 worker. For example, instead of sending the entire historical dataset for a single coin, the data could be split by date, and each worker node could be assigned a specific date range to analyze. This partitioning strategy ensures that each node only works with a manageable subset of the data, preventing memory overflow. Furthermore, given the M1's lower memory bandwidth (67 GB/s vs. 100 GB/s on the M2), it is strategically best suited for tasks that are CPU-bound rather than memory-bandwidth-bound, reinforcing its role as a secondary worker for discrete, less data-intensive calculations.

Feature	Apple M1	Apple M2	Advantage for Trading Bot
Memory Bandwidth	Up to 67 GB/s 58	Up to 100 GB/s 58	Faster data access for the M2, crucial for large datasets and real-time analysis.
Peak FP32 FLOPS	2.29 - 2.61 TFLOPS 58	2.86 - 3.57 TFLOPS 58	Higher computational throughput on the M2 for complex calculations and simulations.
GPU vs. CPU Perf.	GPU is generally faster for parallel tasks.	GPU significantly outperforms CPU for FP32 tasks 58 .	Allows for more aggressive GPU offloading of ML computations on the M2, leading to greater speedups.
Neural Engine	16-core NPU for INT8/FP16 58	16-core NPU for INT8/FP16 58	Highly efficient for low-latency inference after a model has been trained.
RAM (User Device)	8GB (User's device) 58	16GB (User's device) 58	The 16GB M2 is the ideal orchestrator and primary worker; the 8GB M1 is a constrained but usable secondary worker.

By thoughtfully assigning tasks based on these hardware characteristics, it is possible to construct a highly efficient distributed system. The M2 MacBook, with its superior CPU, GPU, and memory bandwidth, should be tasked with the most demanding activities: orchestrating the cluster, ingesting and preprocessing large volumes of data, and training the core predictive models. The M1 MacBook, despite its 8GB RAM bottleneck, can be effectively utilized as a supplementary worker for parallelizable, less data-intensive tasks, such as running individual scenarios in a Monte Carlo simulation or processing smaller chunks of data. This balanced approach maximizes the utility of both machines, creating a synergistic whole that is far more powerful than either could be alone, all within the confines of a strictly free and open-source software environment.

Advanced Predictive Modeling with Open-Source Libraries

At the heart of any sophisticated trading bot lies its predictive model—the algorithm responsible for analyzing market data and generating signals about future price movements. To meet the user's goal of enhanced predictive accuracy and speed, the selection of the right open-source machine learning library is of paramount importance. The ideal candidate must not only be highly accurate but also exceptionally fast, portable across platforms, and capable of leveraging the hardware acceleration offered by Apple's M-Series chips. Based on extensive analysis of available tools, XGBoost emerges as the premier choice for this application, offering a unique combination of state-of-the-art performance, built-in optimizations for speed, and critical support for GPU acceleration [59](#). This section provides a deep dive into the technical merits of XGBoost and explores how its features align perfectly with the constraints and objectives of the project. It also touches upon alternative modeling approaches and the foundational role of data preparation in achieving robust predictive intelligence.

XGBoost, an acronym for eXtreme Gradient Boosting, is an open-source library that implements the gradient boosted decision trees algorithm [59](#). Originally developed in 2014 by Tianqi Chen and Carlos Guestrin, it has since become a cornerstone of the data science community, renowned for its exceptional performance and speed. Its popularity is evidenced by its widespread use in winning solutions for Kaggle competitions, with 17 out of 29 winning solutions in 2015 employing the library [59](#). The library's design philosophy centers on efficiency and scalability, making it an ideal fit for the demands of a real-time trading bot. One of its key technical innovations is the use of second-order gradients, also known as the Hessian, in its optimization process [59](#). By incorporating both first-order (gradient) and second-order (Hessian) derivative information, XGBoost can approximate the loss function with a quadratic function and find its minimum more accurately and in fewer steps than traditional first-order methods like standard stochastic gradient descent. This Newton boosting approach leads to faster convergence and often improved model accuracy [59](#). This mathematical sophistication is a primary driver behind its reputation for being up to 10 times faster than earlier implementations of gradient boosting on a single machine [59](#).

Another critical feature of XGBoost that enhances its suitability for this project is its robust regularization capability [59](#). The library includes a regularized learning objective that explicitly penalizes model complexity. This penalty is applied to two aspects of the model: the number of leaves in the decision trees and the L2 norm of the leaf weights. By

discouraging the creation of overly complex trees, this built-in regularization helps prevent overfitting. An overfit model performs exceptionally well on historical training data but fails to generalize to new, unseen data—a common pitfall in financial forecasting where patterns can be ephemeral. A simpler, more regularized model is more likely to capture true underlying trends rather than random noise, leading to more robust and reliable predictions. This is invaluable in the volatile cryptocurrency markets, where distinguishing signal from noise is the central challenge. The library's active maintenance under the permissive Apache-2.0 license ensures that it continues to evolve and benefit from ongoing community contributions [59](#).

Perhaps the most compelling feature of XGBoost for this specific use case is its native support for GPU acceleration [59](#). The library provides a specific tree construction method called `gpu_hist`, which offloads the most computationally intensive parts of the training process—from histogram pooling to finding the best split—to an NVIDIA GPU [59](#). While the user's hardware is Apple Silicon, this feature highlights a broader principle: modern, high-performance ML libraries are increasingly designed with hardware acceleration in mind. Although XGBoost's direct `gpu_hist` support is for NVIDIA CUDA, the underlying concepts are mirrored in Apple's Metal framework. Many high-level Python libraries that wrap or are inspired by XGBoost's principles, or custom implementations using frameworks like TensorFlow or PyTorch with Metal backends, can achieve similar GPU acceleration on the M2's powerful GPU. The availability of Metal Performance Shaders (MPS) provides pre-optimized kernels for the core linear algebra operations required by gradient boosting, suggesting that a well-tuned implementation could achieve significant speed-ups [58](#). The ability to train complex models on large historical datasets in a fraction of the time is a game-changer, directly contributing to the bot's ability to rapidly analyze market conditions and adapt its strategy. Furthermore, XGBoost's performance extends to distributed systems, allowing it to scale across clusters managed by frameworks like Dask or Spark, which aligns perfectly with the project's distributed computing architecture [59](#).

The efficiency of XGBoost is further bolstered by its custom, memory-efficient data structure known as `DMatrix` [59](#). When data is fed into XGBoost, it is converted into a `DMatrix`, which reorganizes the data in a way that optimizes memory access patterns for the tree-building algorithm. It sorts data by feature, which improves cache locality, handles missing values internally, and can compress sparse data formats [59](#). This pre-processing step accelerates tree-splitting operations, which are the most time-consuming part of the training loop. For a system with constrained resources, like the 8GB RAM M1 laptop acting as a worker node, this memory efficiency is crucial. By minimizing the memory footprint of the dataset during training, the `DMatrix` helps ensure that the

worker node can handle larger chunks of data without exhausting its limited memory, thus improving the stability and performance of the distributed training process. The library is also highly portable, with official support for Linux, Windows, and macOS on both Intel and Apple Silicon architectures, guaranteeing that the chosen solution will work seamlessly across the user's dual-MacBook setup [59](#).

While XGBoost is the recommended primary tool, it is worth considering alternative and complementary modeling approaches. For time-series data, which is inherently sequential, a lightweight one-dimensional Convolutional Neural Network (1D-CNN) could be effective at identifying recurring patterns and trends [21](#). Research has demonstrated the FPGA implementation of such a pipeline for real-time applications, showcasing its potential for low-latency pattern recognition [21](#). On the M2's GPU, a 1D-CNN could be trained and deployed for this purpose. Another emerging frontier is the use of Large Language Models (LLMs) for financial prediction [8](#). Frameworks are being developed to run LLMs locally, allowing them to analyze unstructured data like news articles, social media sentiment, and forum discussions alongside quantitative price data [3](#). A paper on PolySwarm presents a multi-agent LLM framework designed for real-time prediction market trading, hinting at the future direction of AI in this domain [30](#). However, running state-of-the-art LLMs locally on consumer hardware remains challenging, though the trend towards more efficient models and open-source frameworks suggests this will become more feasible [13](#). For the current project, a hybrid approach could be envisioned: using XGBoost for its strength in tabular, quantitative data, while exploring a smaller, fine-tuned LLM for sentiment analysis if computational resources permit. The Helix 1.0 framework, an open-source tool for reproducible ML on tabular scientific data, could also be valuable for ensuring the transparency and robustness of the modeling process [10](#).

Ultimately, the success of the predictive modeling layer depends on the quality of the input data and the engineering of relevant features. Raw price and volume data is rarely sufficient. The data preparation stage is where domain expertise is most critical. Features must be engineered to capture the nuances of market behavior. These features could include:

- **Technical Indicators:** Standard indicators like Moving Averages (SMA, EMA), Relative Strength Index (RSI), Bollinger Bands, MACD, and various volatility measures.
- **Lagging and Leading Indicators:** Features that look backward (lagging) to identify established trends and features that look forward (leading) to anticipate reversals.

- **Order Book Imbalance:** If Level 2 data is available, features derived from the bid-ask spread and the relative depth of buy vs. sell orders can be powerful predictors.
- **Time-Based Features:** Hour of the day, day of the week, and seasonality effects can capture cyclical patterns in market activity.
- **Cross-Asset Correlations:** Features representing the correlation between the target cryptocurrency and other assets (e.g., BTC, ETH, major indices) can provide additional context.

These features are then fed into the XGBoost model, which learns the complex, non-linear relationships between them and the future price movement. The iterative process of feature engineering, model training, validation, and hyperparameter tuning is a continuous cycle that is essential for maintaining the bot's predictive accuracy in a constantly evolving market. The speed afforded by the Apple Silicon hardware and the distributed Dask cluster allows this cycle to happen much faster, enabling the developer to experiment and refine the model with agility.

Feature / Capability	Description	Benefit for Trading Bot
Second-Order Gradients	Uses both gradient and Hessian information for optimization. 59	Enables faster convergence and improved accuracy compared to first-order methods.
Built-in Regularization	Penalizes model complexity (number of leaves, leaf weights). 59	Prevents overfitting, leading to more robust and generalizable models.
GPU Acceleration Support	Offloads intensive computations to a GPU (e.g., via <code>gpu_hist</code>). 59	Drastically reduces training time on large datasets, leveraging the M2's powerful Metal-accelerated GPU.
Memory-Efficient Data Matrix	Custom data structure that optimizes memory layout and handles sparsity. 59	Reduces memory footprint, improving stability and performance on memory-constrained machines like the M1.
High Portability	Officially supports Linux, Windows, and macOS (Intel & Apple Silicon). 59	Ensures the code runs seamlessly across the user's dual-MacBook setup.
Distributed Training	Integrates with frameworks like Dask and Spark. 59	Allows for scaling model training across the distributed cluster of two MacBooks.

In summary, XGBoost stands out as the optimal open-source library for the predictive modeling component of the trading bot. Its combination of mathematical sophistication, built-in regularization, and critical support for hardware acceleration directly addresses the project's core requirements for speed and accuracy. By implementing XGBoost within the distributed Dask framework on the dual-MacBook cluster, the user can build a powerful and efficient system for developing, training, and deploying high-performance predictive models tailored to the complexities of the cryptocurrency market.

Accelerating Backtesting and Scenario Simulation

The user's ambition to "analyse all the future position and then selects the best one" is a computationally intractable problem that cannot be solved by brute-force enumeration of every possible market outcome. Instead, this goal must be interpreted as the need for a rapid and intelligent evaluation of a vast number of potential future scenarios to identify the trade with the highest expected value. This is precisely the domain of backtesting and Monte Carlo simulation, which are indispensable tools in quantitative finance for validating trading strategies and managing risk. However, these processes are notoriously slow, often taking hours or even days to run on a single machine, especially when dealing with high-frequency data and complex strategies. To overcome this bottleneck and achieve the desired speed, the process must be massively parallelized. This section details how to implement a distributed backtesting and scenario simulation framework using the open-source Python library Dask, transforming the two MacBooks into a powerful computational engine capable of executing thousands of simulations concurrently. This approach directly enables the bot to "rapidly analyze" potential futures and make informed decisions in near real-time.

The core of this acceleration strategy is Dask, a flexible parallel computing library for analytic workflows [50](#). Dask is designed to scale computations from a single machine to a distributed cluster, and it integrates seamlessly into the existing Python ecosystem, working with popular libraries like NumPy, Pandas, and scikit-learn. For the trading bot, Dask will be used to break down the monolithic task of backtesting a single strategy over a long period into thousands of smaller, independent tasks. For example, instead of running one backtest from January 2020 to December 2023, Dask can divide this period into daily or hourly chunks. Each chunk can then be treated as an independent job. The Dask scheduler, running on the M2 MacBook, will distribute these jobs to the available worker nodes—in this case, both the M2 and the M1 MacBooks. Each machine will execute its assigned backtest simulation in parallel. Once all the small backtests are complete, the Dask client on the M2 will collect all the results, combine them, and produce a single, comprehensive report as if one giant backtest had been run. This parallelization can yield near-linear speedups depending on the number of independent tasks and the efficiency of the hardware, dramatically reducing the wall-clock time required for analysis [36](#).

The implementation of this distributed backtesting framework follows a clear, logical flow. First, the historical market data must be prepared in a format that is amenable to chunking. Storing the data in a columnar format like Apache Arrow or Parquet is highly recommended, as these formats are designed for efficient splitting and reading of data

subsets. The data would be loaded into a Dask DataFrame, which behaves like a Pandas DataFrame but operates on data that may be larger than memory and is processed in parallel. Next, the backtesting logic itself must be written as a pure function. This function should take a specific time slice (e.g., a single day of data) as input, initialize a portfolio object, iterate through the ticks or bars in that slice, apply the trading strategy's rules, and return a structured output containing performance metrics for that period (e.g., P&L, Sharpe ratio, max drawdown). This function is then mapped over the chunks of the Dask DataFrame. Dask handles the rest of the process: dividing the DataFrame into partitions, sending each partition to a worker node for processing, and gathering the results. This modular design keeps the code clean and testable, and it allows for easy experimentation with different strategies by simply changing the backtesting function.

Beyond validating a single strategy, this parallelized framework is perfectly suited for conducting extensive walk-forward analysis and Monte Carlo simulations, which are crucial for robust strategy development. Walk-forward analysis involves repeatedly training a model on a historical "in-sample" period and testing it on a subsequent "out-of-sample" period, sliding this window forward through time. This process can be parallelized by treating each fold of the cross-validation as an independent task. Monte Carlo simulations, used to "analyze all potential future positions," can be implemented by introducing randomness into the simulation. For example, one could run thousands of simulations where the future price path is generated by sampling from a probability distribution (e.g., a normal distribution of returns with a certain mean and volatility). Each simulated path represents a potential future. The trading bot's logic is then executed on each of these thousands of paths in parallel. The results from all simulations are aggregated to produce a probability distribution of potential outcomes, such as final portfolio value or profit-and-loss. From this distribution, the bot can calculate the expected value of a trade, its Value-at-Risk (VaR), and other risk metrics. This allows for a much more nuanced decision-making process than simply looking at the performance of a single deterministic backtest. The ability to generate and analyze this rich probabilistic forecast in a short amount of time is a direct result of leveraging the parallel processing power of the two-MacBook cluster via Dask [11](#).

The division of labor between the M2 and M1 MacBooks within this Dask cluster is critical for efficiency. The M2, with its 16GB of RAM and more powerful CPU/GPU, should be configured as the Dask scheduler and also as a worker node. The scheduler is responsible for monitoring the cluster's health, managing the task graph, and distributing work to the workers. Having the M2 act as both ensures that the most powerful machine is managing the entire operation. The M1, with its 8GB of RAM, should be added as another worker node. Care must be taken to ensure that the data chunks assigned to the M1 are sized appropriately to fit within its memory limit. For instance, if the total

historical dataset is 10GB, a chunk size of around 1-2GB would be safe for the M1 worker. The Dask dashboard, a web-based diagnostic tool, can be used to monitor the cluster's performance, visualize the task stream, and identify any imbalances where one worker is overloaded while the other is idle. This visibility is crucial for tuning the system and ensuring optimal resource utilization. The setup can be managed entirely through configuration files or command-line arguments when starting the Dask processes on each machine, requiring no paid commercial software [50](#).

The table below illustrates how the backtesting and simulation tasks would be allocated across the two machines in the Dask cluster.

Task Type	Description	Allocation Strategy	Rationale
Backtesting	Validating a trading strategy against historical data.	Chunks of historical data are divided and assigned to worker nodes.	Parallel processing of independent time periods drastically reduces total runtime. 50
Walk-Forward Analysis	Repeatedly training and testing a model on sliding time windows.	Each fold of the cross-validation process is treated as an independent task.	Enables robust validation of dynamic models without a prohibitive computational cost.
Monte Carlo Simulation	Generating thousands of potential future price paths to assess risk and reward.	Each simulated price path is executed as a separate task.	Provides a probabilistic forecast of outcomes, allowing for data-driven risk management. 11
Cluster Management	Scheduling, monitoring, and collecting results.	The M2 MacBook acts as the central scheduler.	The M2's superior processing power and larger memory make it the ideal choice for the coordinator role. 58
Parallel Execution	Running the assigned tasks on the worker nodes.	The M2 acts as a worker, and the M1 acts as a dedicated worker.	Utilizes all available hardware resources. The M1's 8GB RAM requires careful chunk sizing to avoid memory issues. 58

By implementing this Dask-based framework, the trading bot gains a powerful analytical engine. The ability to rapidly run thousands of backtests and simulations empowers the bot to move beyond simple signal generation and engage in sophisticated scenario planning. It can evaluate not just if a trade is profitable on average, but also understand the distribution of potential profits and losses, the likelihood of large drawdowns, and how the strategy might behave under different market regimes. This deeper level of analysis is the key to selecting the "best one" among a multitude of possibilities, not by guessing, but by making a decision based on a comprehensive and rapidly computed assessment of all potential futures. This approach directly fulfills the user's vision of a bot that is both prescient and quick.

Strategic Use of Free Cloud Infrastructure for Automation and Data Access

While the core computational engine of the trading bot resides on the user's local hardware, leveraging strategically selected free cloud services can provide significant, cost-free enhancements to its functionality. These services are not intended to replace the powerful local cluster but to augment it by handling specific, non-computational tasks with high reliability and low latency. The primary areas for augmentation are automated data ingestion and the establishment of a low-latency data pipeline. By using services like Cloudflare Workers and GitHub Actions, which offer generous free tiers, the bot can be kept continuously supplied with fresh market data and its operational workflows can be automated, freeing up local resources and ensuring the system operates efficiently without manual intervention. This section explores how to integrate these free cloud tools into the trading bot's architecture, detailing their specific capabilities and usage patterns to stay strictly within their cost-free limits.

Cloudflare Workers presents a unique opportunity for automating data-related tasks at the edge of the internet, with a free tier that is surprisingly robust for small-scale applications [39](#) [60](#). The most valuable feature for this project is "Cron Triggers," which allows a piece of JavaScript or TypeScript code (a Worker) to be scheduled to run on a timed interval, completely free of charge [62](#) [63](#). This feature is ideal for automating the periodic collection of market data from various cryptocurrency exchanges. A custom Worker can be written to contain the logic for calling specific endpoints on an exchange's API to fetch the latest ticker prices, order book snapshots, or historical candle data. This Worker can then be configured with a cron schedule, for example, to run every minute (`<em>/1 </em> <em> </em> <em></em>`) or every five minutes (`</em>/5 <em> </em> <em> </em>`), to ensure the local bot is always working with up-to-date information [62](#). The execution of these cron-triggered Workers happens on Cloudflare's global network of data centers, which means the code runs close to the origin of the data (the exchange APIs), potentially reducing network latency [61](#). It is important to note that Cron Triggers do not run in a fixed datacenter; they execute on whatever PoP (Point of Presence) has the lowest load at the time of scheduling, which is a mechanism Cloudflare uses to efficiently utilize its massive infrastructure [64](#). While this introduces some variability in the exact location, the overall performance benefit from the global network is significant.

To manage the cron triggers, a Wrangler configuration file can be used to define the schedules, which are then deployed along with the Worker code [62](#). A single Worker script can even be configured with multiple cron triggers to perform different actions on

different schedules, although there is a limit of three triggers per individual Worker script [63](#). For example, one trigger could fetch tick data every minute, while another, running once an hour, could fetch larger historical datasets for model retraining. The data fetched by the Cloudflare Worker can be handled in several ways. It could be formatted as JSON and sent via an HTTP POST request to a private endpoint on the local M2 machine, which would have a script listening for this data and saving it to a local file or database. Alternatively, it could be stored in a more durable backend, but given the "absolute free" constraint, the most straightforward approach is direct delivery to the local machine. The free tier of Cloudflare Workers comes with strict but adequate limits for this purpose: 100,000 requests per day, 10 milliseconds of CPU time per invocation, and 128 MB of memory per function call [60](#). These limits are more than sufficient for the light-weight task of making an API call and parsing a response, as the actual computation is minimal. Exceeding the daily request limit results in a **1027** error, so the frequency of the cron jobs must be planned accordingly [60](#).

Complementing the scheduled data pulls from Cron Triggers, Cloudflare's support for WebSockets offers another avenue for low-latency data access [66](#) [67](#). Many cryptocurrency exchanges provide real-time market data streams via WebSockets. A Cloudflare Worker can be configured to act as a WebSocket client, connecting to an exchange's streaming endpoint. The Worker would then receive a continuous flow of data (e.g., price updates, trades) and can forward this information to the local bot. This approach can be more efficient than polling APIs every second, as it eliminates delays associated with request-response cycles. The Worker can parse the incoming WebSocket messages and push the relevant data points to the local machine via another WebSocket connection or a simple HTTP request. This creates a persistent, bi-directional communication channel that keeps the local bot updated with the latest market movements in near real-time. Again, the 10ms CPU time limit per invocation applies to the initial setup of the WebSocket connection, but the ongoing message handling is a separate, long-running process that is counted differently and fits within the free tier's allowances for long-running agent workflows [60](#) [61](#). By combining scheduled data pulls for bulk data with a WebSocket bridge for real-time ticks, the bot can be supplied with a comprehensive and timely data feed.

GitHub Actions serves as the second pillar of this free cloud infrastructure, providing a powerful platform for automating the entire software development lifecycle, including the bot's operational workflows [35](#). For public repositories, GitHub Actions offers a generous free tier that includes a monthly quota of 2,000 job minutes for the free plan, which is ample for many automation tasks [44](#). The primary use case for the trading bot is to automate the deployment and execution of its components. For instance, a workflow

can be triggered on a schedule (using a `cron` trigger in the YAML file) to run the backtesting module on the local cluster. This ensures that the strategy is regularly validated against the latest data without requiring a developer to manually initiate the process. Another powerful use case is continuous integration: a workflow can be configured to run automatically whenever new code is pushed to the main branch of the bot's GitHub repository. This workflow could lint the code, run unit tests, and then trigger a full backtest, providing immediate feedback if a change has negatively impacted performance. This practice of "testing in production" (in a controlled, automated manner) is a hallmark of robust software development.

Furthermore, GitHub Actions can be used to automate the deployment of the bot's code to the local MacBooks. The M2, as the central node, could run a self-hosted runner, which is a special program that you host yourself to run jobs from your GitHub repository [54](#). While GitHub recently announced a new pricing model for self-hosted runners (**0.002/minute starting March 2026**), the base runner software itself is free, and the macOS runner costs 0.062 per minute, whereas a slim Linux runner costs only \$0.002 per minute [46](#). This pricing disparity strongly suggests that the core logic of the trading bot, including the Dask client and data processing scripts, should be containerized using Docker and run on a Linux-based runner. This avoids the high cost of macOS runners for long-running or frequent jobs. The containerization also ensures that the development environment on the MacBooks closely mirrors the production-like environment on the Linux runner, promoting consistency and portability.

Service	Primary Function	Key Free Tier Feature	Usage Example for Trading Bot	Cost Constraint
Cloudflare Workers	Serverless Functions & Automation	Cron Triggers for scheduled execution 62	Schedule a Worker to fetch market data from exchanges every minute and push it to the local M2 machine.	No payment method needed; stays within daily request and CPU time limits 60 63 .
Cloudflare Workers	Real-time Data Streaming	WebSocket API support 66	Act as a WebSocket client to an exchange's API, relay real-time tick data to the local bot.	Long-running agent workflows are billed based on usage above free tier limits, but basic streaming fits within allowances 61 .
GitHub Actions	Continuous Integration & Deployment	Free minutes for public/private repositories 44	Automate backtesting runs on a schedule or on every code push to validate strategy performance.	Must stay within monthly minute quotas; prefer cheaper Linux runners over expensive macOS runners 46 .
GitHub Self-Hosted Runners	On-Premise CI/CD Execution	Runner software is free to install and use 54	Deploy the bot's code from GitHub to the local MacBooks automatically using <code>rsync</code> or <code>scp</code> within a workflow.	New pricing model charges per minute (\$0.002/min); use strategically and on-demand to minimize cost 45 .

In conclusion, a strategic and judicious use of free cloud services can significantly enhance the capabilities of a locally-run trading bot. Cloudflare Workers provides a cost-

free mechanism for automated, low-latency data ingestion, keeping the local models supplied with fresh information. GitHub Actions offers a powerful, free (within limits) platform for automating backtesting, code validation, and deployment workflows. By integrating these tools, the bot becomes more autonomous, reliable, and maintainable. This hybrid approach, which combines the raw power of local hardware with the convenience and automation of the cloud, is the most effective way to build a sophisticated trading system without incurring any subscription or usage fees.

Synthesis and Final Recommendations

The objective of creating a high-speed, high-accuracy cryptocurrency trading bot using only free resources is achievable through a meticulously designed, multi-layered architecture that synergistically combines local hardware, open-source software, and strategically selected cloud services. The analysis culminates in a concrete, actionable framework that directly addresses the user's need to "analyse all the future position and then selects the best one faster and quicker" at "absolute free" cost. This final section synthesizes the preceding findings into a cohesive recommendation, outlining a complete architectural blueprint and highlighting the critical considerations and potential challenges that must be navigated to successfully implement this system. The proposed solution is not a single product but a composite system built from best-in-class free components, orchestrated to deliver maximum performance within the specified constraints.

The recommended architecture is a hybrid model centered on a distributed computing cluster formed by the user's two MacBooks. The M2-equipped MacBook (16GB RAM) will serve as the Central Processing Node (CPN), responsible for orchestrating the entire operation, ingesting bulk market data, training the core predictive models, and executing the final trade. The M1-equipped MacBook (8GB RAM) will function as a Worker Node (WN), specializing in the parallel execution of computationally intensive but memory-constrained tasks, such as individual simulations within a larger backtest. Communication and task distribution between these two machines will be managed by the open-source Python library Dask, which provides a seamless and efficient framework for parallel processing across a local network [50](#). This division of labor ensures that the most powerful machine handles the most complex logic, while the less powerful one contributes valuable processing cycles, creating a balanced and efficient cluster.

For the core predictive intelligence, the primary algorithmic engine should be XGBoost, an open-source library implementing gradient boosted decision trees [59](#). Its superiority stems from a combination of factors: its use of second-order gradients for faster convergence, built-in L1/L2 regularization to prevent overfitting, and, most critically, its support for GPU acceleration [59](#). By leveraging the M2's powerful Metal-accelerated GPU, the computationally expensive process of training the model on historical data can be completed in a fraction of the time required by a CPU-only approach. This directly translates to the "faster" aspect of the user's goal. The model will be trained on a rich set of engineered features derived from market data, and its predictions will form the basis of the bot's trading signals. For analyzing potential future outcomes, the Dask cluster will be used to run extensive Monte Carlo simulations. Thousands of potential future price paths will be generated and evaluated in parallel, providing a probabilistic forecast of outcomes that allows the bot to select the trade with the highest expected value, thus fulfilling the "analyse all future positions" requirement in a computationally feasible manner.

To augment this powerful local cluster, two free cloud services should be integrated. First, Cloudflare Workers, with its free Cron Trigger feature, will be used to automate the periodic collection of market data from exchange APIs [62](#) [63](#). This offloads the scheduling and basic fetching logic from the local machines, ensuring the bot is always supplied with fresh data without manual intervention. Second, GitHub Actions will be employed to create a continuous integration and deployment (CI/CD) pipeline [44](#). This pipeline will automate critical workflows, such as running backtests on a regular schedule or after every code change, and will facilitate the automatic synchronization of the bot's codebase across both MacBooks. This ensures the system remains robust, validated, and consistent. It is imperative to adhere to the free tier limitations of these services; for instance, preferring cheaper Linux runners in GitHub Actions over expensive macOS runners will be crucial for long-term cost-free operation [46](#).

Despite the feasibility of this architecture, several significant challenges and constraints must be acknowledged and actively managed. The 8GB RAM on the M1 MacBook is the most pressing hardware limitation. It will act as a hard ceiling on the size of the data chunks that can be processed by the worker node. Careful data partitioning and memory management within the Dask framework will be essential to prevent crashes and ensure stability. The performance of the entire distributed system is also dependent on the speed and reliability of the local network connection between the two MacBooks. A wired Ethernet connection is strongly recommended over Wi-Fi to minimize communication latency between the CPN and the WN. Finally, the implementation complexity should not be underestimated. Setting up a robust Dask cluster, integrating GPU-accelerated

libraries, containerizing the application for cross-platform compatibility, and managing the CI/CD pipelines are all advanced technical tasks that require a solid foundation in software engineering and system administration.

In conclusion, the path to building an optimized, zero-cost trading bot is paved with a combination of powerful local hardware, efficient open-source software, and clever use of free cloud services. The recommended blueprint provides a clear roadmap for achieving this goal. By establishing a Dask cluster with the M2 as the orchestrator and the M1 as a worker, leveraging XGBoost's GPU-accelerated speed for predictive modeling, and using Cloudflare and GitHub Actions for automated data and workflow management, the user can construct a system that is both fast and analytically robust. Success will depend on diligent attention to the system's constraints, particularly the memory limits of the M1, and a commitment to disciplined software development practices. If these challenges are met head-on, the resulting trading bot will be a testament to the power of modern, freely available technologies in the pursuit of quantitative trading excellence.

Reference

1. Earnings call transcript: Cloudflare beats Q1 2026 estimates, stock ... <https://www.investing.com/news/transcripts/earnings-call-transcript-cloudflare-beats-q1-2026-estimates-stock-rises-93CH-4670970>
2. [PDF] Impact Report 2025 - Cloudflare https://cf-assets.www.cloudflare.com/slt3lc6tev37/7koyyovVxIqK8zdG1pqo6O/232476d0cc224d793025546295e68b20/Impact-Report-2025_01282026.pdf
3. The Case for Local AI Has Never Been Stronger | HackerNoon <https://hackernoon.com/the-case-for-local-ai-has-never-been-stronger>
4. [PDF] Autonomous Agents on Blockchains: Standards, Execution Models ... <https://arxiv.org/pdf/2601.04583>
5. Towards Secure Systems of Interacting AI Agents - arXiv <https://arxiv.org/html/2505.02077v2>
6. Computer Science - arXiv <https://www.arxiv.org/list/cs/new?skip=25&show=2000>
7. From Logic Monopoly to Social Contract: Separation of Power and ... <https://arxiv.org/html/2603.25100v1>
8. LLM-as-a-Prophet: Understanding Predictive Intelligence ... - arXiv <https://arxiv.org/html/2510.17638v1>

9. Machine Learning Jul 2025 - arXiv <https://www.arxiv.org/list/cs.LG/2025-07?skip=2225&show=1000>
10. Artificial Intelligence Jul 2025 - arXiv <https://www.arxiv.org/list/cs.AI/2025-07?skip=2100&show=2000>
11. Computer Science - arXiv <https://www.arxiv.org/list/cs/new?skip=450&show=1000>
12. Markus864 - TikTok <https://www.tiktok.com/@markus864>
13. Optimize Local AI on 128GB Mac: Top Models & Limitations - LinkedIn https://www.linkedin.com/posts/jonanthony_localai-applesilicon-macstudio-activity-7434332157143248897-MI66
14. cloud-20241231 - SEC.gov <https://www.sec.gov/Archives/edgar/data/1477333/000147733325000043/cloud-20241231.htm>
15. Free Cloud Computing Services - AWS Free Tier <https://aws.amazon.com/free/>
16. AWS Free Tier FAQs - How to use AWS Cloud Services for Free <https://aws.amazon.com/free/registration-faqs/>
17. Learn why the AWS Free Tier requires a payment method <https://repost.aws/knowledge-center/free-tier-payment-method>
18. AWS Free Tier FAQs <https://aws.amazon.com/free/free-tier-faqs/>
19. Is AWS Free for 12 Months in 2026? AWS Free Tier Explained <https://www.fusion-institute.com/is-aws-free-for-12-months-in-2026-aws-free-tier-explained>
20. #ai #embeddedsystems #esp32 #ollama #opensource ... - LinkedIn https://www.linkedin.com/posts/mercy-jemeli-b34a86249_ai-embeddedsystems-esp32-activity-7424338462104817664-a6uU
21. Electronics, Volume 15, Issue 3 (February-1 2026) – 226 articles <https://www.mdpi.com/2079-9292/15/3>
22. [PDF] Some Simple Economics of AGI - arXiv <https://arxiv.org/pdf/2602.20946>
23. Deploying a project to fly.io with GitHub Actions - DEV Community <https://dev.to/ruthmoog/deploying-a-project-to-flyio-with-github-actions-2c7e>
24. [PDF] PROviding Computing solutions for ExaScale ChallengeS <https://www.ec.europa.eu/research/participants/documents/downloadPublic?documentIds=080166e5bca6fd5f&appId=PPGMS>
25. NixOS - DistroWatch.com <https://distrowatch.com/table-mobile.php?distribution=nixos&pkglist=true&version=unstable>
26. Troubleshooting Common Development Errors | PDF - Scribd <https://www.scribd.com/document/752533028/cektitle>
27. Computing Science, Communication and Security - Springer Nature <https://link.springer.com/content/pdf/10.1007/978-981-15-6648-6.pdf>

28. [PDF] Layer7 API Gateway 11.1 - Broadcom TechDocs <https://techdocs.broadcom.com/content/dam/broadcom/techdocs/us/en/pdf/ca-enterprise-software/layer7-api-management/api-gateway/11-0/Layer7-api-gateway-11-1.pdf>
29. Render vs Fly.io (2026): Simple PaaS vs Global Edge Containers <https://www.buildmvpfast.com/compare/render-vs-fly-io>
30. PolySwarm: A Multi-Agent Large Language Model Framework for ... <https://arxiv.org/html/2604.03888v1>
31. QORECHAIN Quantum Safe AI Optimized Interchain Architecture https://www.academia.edu/144643193/QORECHAIN_Quantum_Safe_AI_Optimized_Interchain_Architecture
32. Tech Tools For Activism - Pentesting - Penetration Testing - Hacking <https://www.scribd.com/document/335681070/Tech-Tools-For-Activism-Pentesting-Penetration-Testing-Hacking-OpNewBlood>
33. Profile for GitHub - Linknovate <https://www.linknovate.com/affiliation/github-435897/all/?query=valid%20token>
34. Profile for YouTube - Linknovate <https://www.linknovate.com/affiliation/youtube-2961559/all/?query=proxy%20server%20computer%20passes>
35. Terraform is free - ArgoCD & Flux are... | Pester Mbhetse - LinkedIn https://www.linkedin.com/posts/pester-mbhetse_be-ready-for-2026-no-excuses-left-activity-7450811977041330176-BkPz
36. Algorithms and Architectures for Parallel Processing - Springer Nature <https://link.springer.com/content/pdf/10.1007/978-981-96-1542-1.pdf>
37. [PDF] Intelligent Sustainable Systems - ResearchGate https://www.researchgate.net/profile/Jasleidy-Prada-Segura/publication/379686998_Strategy_for_Continuous_Training_and_Methodological_Exchange_for_Public_Accounting_Students_Through_the_I-Tournament_Tool_and_Its_Contribution_to_Management_for_the_Business_Sector/links/666611f7de777205a31f7446/Strategy-for-Continuous-Training-and-Methodological-Exchange-for-Public-Accounting-Students-Through-the-I-Tournament-Tool-and-Its-Contribution-to-Management-for-the-Business-Sector.pdf
38. SIGBOVIK 2025 Proceedings Overview | PDF | Letter Case - Scribd <https://www.scribd.com/document/879154885/Proceedings>
39. Pricing · Cloudflare Workers docs <https://developers.cloudflare.com/workers/platform/pricing/>
40. 3 - CodaLab Worksheets <https://worksheets.codalab.org/rest/bundles/0xd74f36104e7244e8ad99022123e78884/contents/blob/frequent-classes>
41. ChatForest — AI agents reviewing AI tools. Honestly. <https://chatforest.com/>

42. Crypto Arbitrage Bots and Machine Learning: How They Actually Work <https://alphagaindaily.com/en/blog/crypto-arbitrage-bot-machine-learning-guide>
43. Machine learning approaches to predicting energy price correlation <https://www.sciencedirect.com/science/article/pii/S0040162525005463>
44. GitHub Actions billing <https://docs.github.com/billing/managing-billing-for-github-actions/about-billing-for-github-actions>
45. Corey Quinn's Post - LinkedIn https://www.linkedin.com/posts/coquinn_github-just-announced-theyre-charging-0002-activity-7406817983098118144-zoGl
46. Actions runner pricing - GitHub Docs <https://docs.github.com/en/billing/reference/actions-runner-pricing>
47. LINUX Unplugged - TopPodcast.com https://toppodcast.com/podcast_feeds/linux-unplugged/
48. Simple Index - NJU Mirror <https://mirror.nju.edu.cn/pypi/web/simple>
49. Udemyfor Business Course List | PDF | Cloud Computing - Scribd <https://www.scribd.com/document/607379618/UdemyforBusinessCourseList>
50. Setup — dask 0.16.1 documentation - Read the Docs <http://dask-local.readthedocs.io/en/latest/setup.html>
51. Computer Science - arXiv <https://arxiv.org/list/cs/new>
52. 人工智能2025_12_19 - arXiv每日学术速递 <https://www.arxivdaily.com/thread/74929>
53. Agents llms-full.txt - Cloudflare Docs <https://developers.cloudflare.com/agents/llms-full.txt>
54. Self-hosted runners - GitHub Docs <https://docs.github.com/actions/hosting-your-own-runners>
55. Balíky softvéru v „sid“, Subsekcia python - Debian -- Packages <https://packages.debian.org/sk/sid/python/>
56. Debian -- Software Packages in "trixie", Subsection python <https://packages.debian.org/trixie/python/>
57. Debian -- Software Packages in "sid", Subsection python <https://packages.debian.org/sid/python/>
58. Evaluating the Apple Silicon M-Series SoCs for HPC Performance ... <https://arxiv.org/html/2502.05317v1>
59. Ultimate guide to XGBoost library in Python - Deepnote <https://deepnote.com/blog/ultimate-guide-to-xgboost-library-in-python>
60. Limits · Cloudflare Workers docs <https://developers.cloudflare.com/workers/platform/limits/>
61. Cloudflare Workers <https://workers.cloudflare.com/>

- 62. Cron Triggers - Workers - Cloudflare Docs <https://developers.cloudflare.com/workers/configuration/cron-triggers/>
- 63. Introducing Cron Triggers for Cloudflare Workers <https://blog.cloudflare.com/introducing-cron-triggers-for-cloudflare-workers/>
- 64. Where do Cron-triggered workers run? - Cloudflare Community <https://community.cloudflare.com/t/where-do-cron-triggered-workers-run/282128>
- 65. Multiple Cron Triggers · Cloudflare Workers docs <https://developers.cloudflare.com/workers/examples/multiple-cron-triggers/>
- 66. Using the WebSockets API · Cloudflare Workers docs <https://developers.cloudflare.com/workers/examples/websockets/>
- 67. WebSockets · Cloudflare Network settings docs <https://developers.cloudflare.com/network/websockets/>